



Digital Design

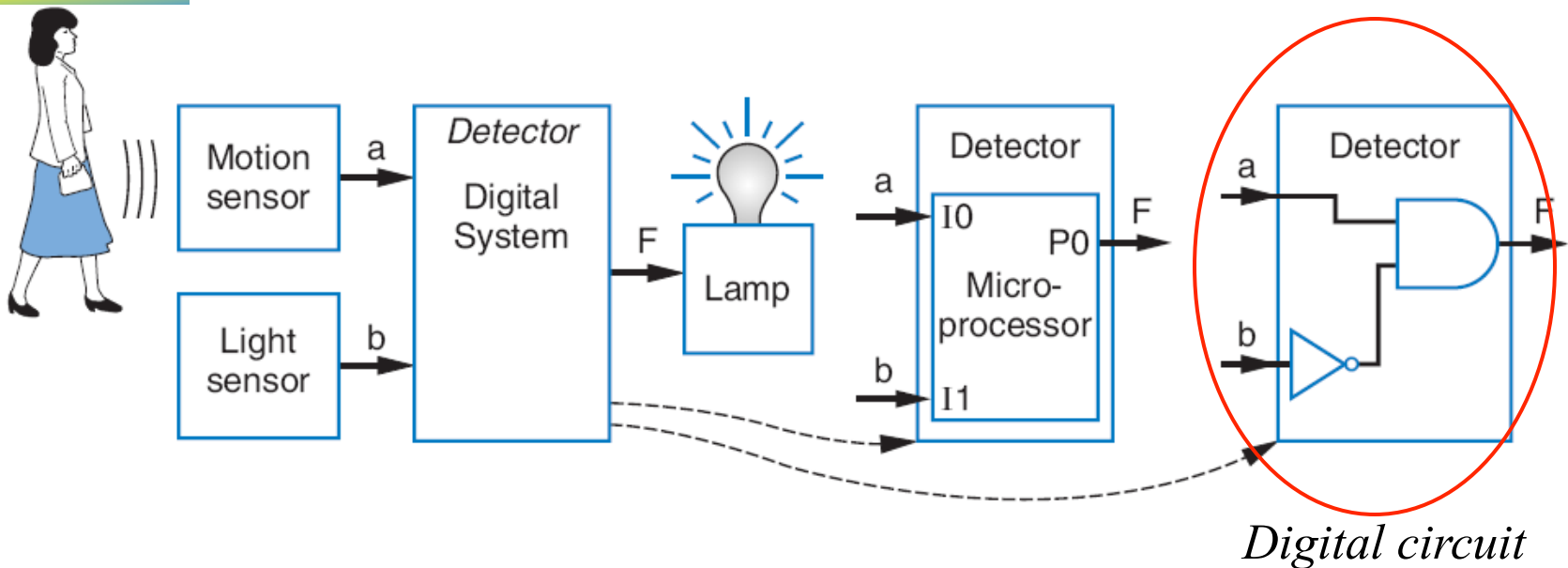
Chapter 2: Combinational Logic Design

Slides to accompany the textbook *Digital Design*, First Edition,
by Frank Vahid, John Wiley and Sons Publishers, 2007.
<http://www.ddvahid.com>

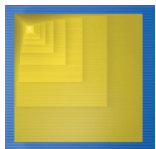
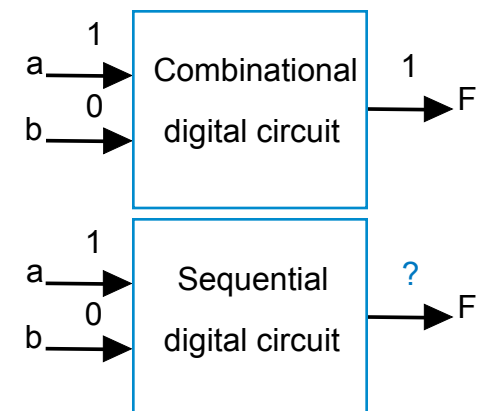
Copyright © 2007 Frank Vahid

Instructors of courses requiring Vahid's *Digital Design* textbook (published by John Wiley and Sons) have permission to modify and use these slides for customary course-related activities, subject to keeping this copyright notice in place and unmodified. These slides may be posted as unanimated pdf versions on publicly-accessible course websites.. PowerPoint source (or pdf with animations) may not be posted to publicly-accessible websites, but may be posted for students on internal protected sites or distributed directly to students by other electronic means. Instructors may make printouts of the slides available to students for a reasonable photocopying charge, without incurring royalties. Any other use requires explicit permission. Instructors may obtain PowerPoint source or obtain special use permissions from Wiley – see <http://www.ddvahid.com> for information.

Introduction

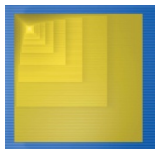
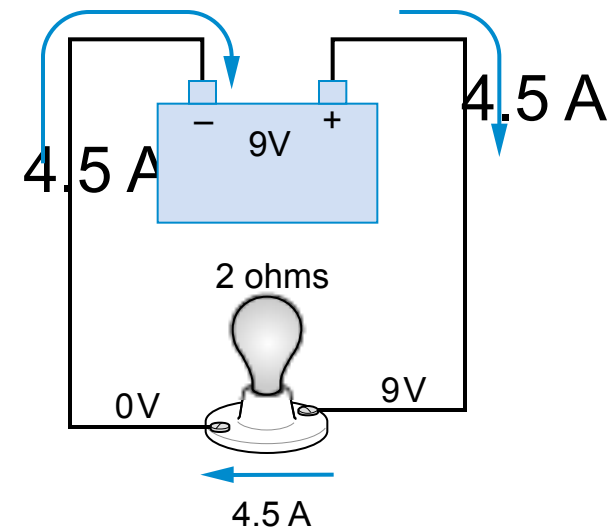


- Let's learn to design digital circuits
- We'll start with a simple form of circuit:
 - **Combinational circuit**
 - A digital circuit whose outputs depend solely on the present combination of the circuit inputs' values



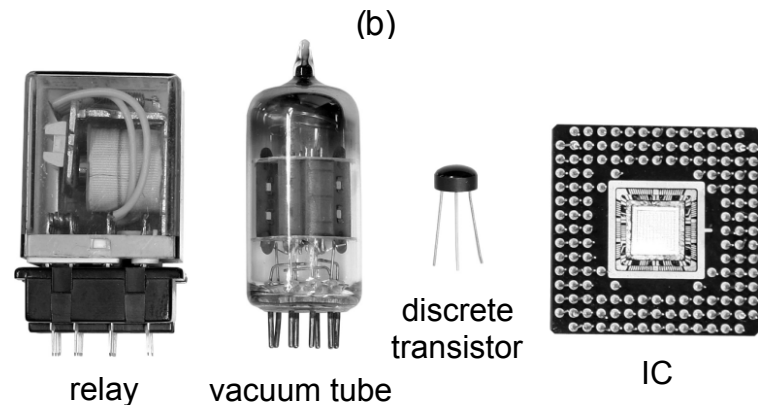
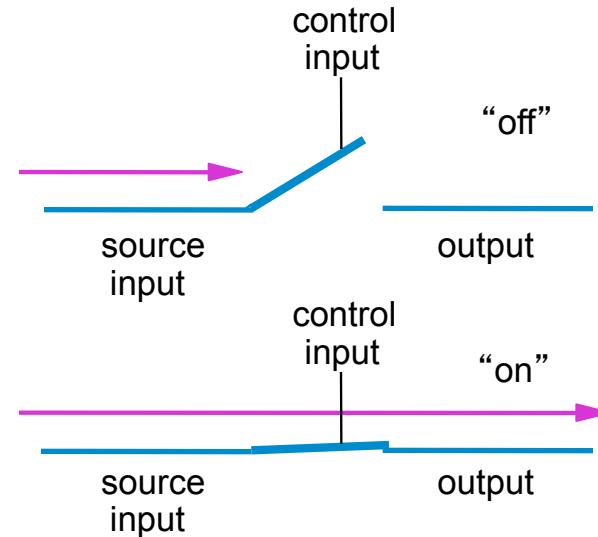
Switches

- Electronic switches are the basis of binary digital circuits
 - Electrical terminology
 - **Voltage**: Difference in electric potential between two points
 - Analogous to water pressure
 - **Current**: Flow of charged particles
 - Analogous to water flow
 - **Resistance**: Tendency of wire to resist current flow
 - Analogous to water pipe diameter
 - $V = I * R$ (Ohm's Law)

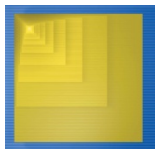


Switches

- A switch has three parts
 - Source input, and output
 - Current wants to flow from source input to output
 - Control input
 - Voltage that controls whether that current can flow
- The amazing shrinking switch
 - 1930s: Relays
 - 1940s: Vacuum tubes
 - 1950s: Discrete transistor
 - 1960s: Integrated circuits (ICs)
 - Initially just a few transistors on IC
 - Then tens, hundreds, thousands...

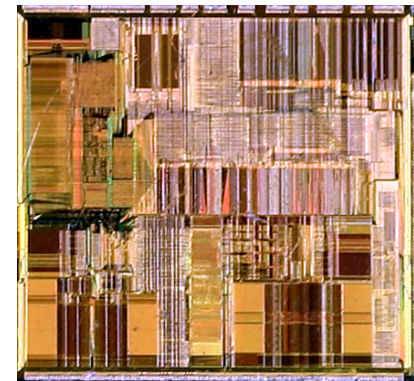
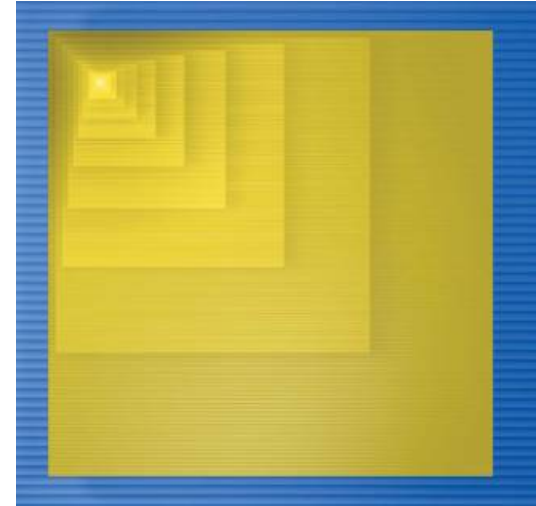


quarter
(to see the relative size)

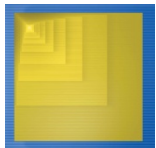


Moore's Law

- IC capacity doubling about every 18 months for several decades
 - Known as “Moore's Law” after Gordon Moore, co-founder of Intel
 - Predicted in 1965 predicted that components per IC would double roughly every year or so
 - Book cover depicts related phenomena
 - For a particular number of transistors, the IC shrinks by half every 18 months
 - Notice how much shrinking occurs in just about 10 years
 - Enables incredibly powerful computation in incredibly tiny devices
 - Today's ICs hold *billions* of transistors
 - The first Pentium processor (early 1990s) needed only 3 million

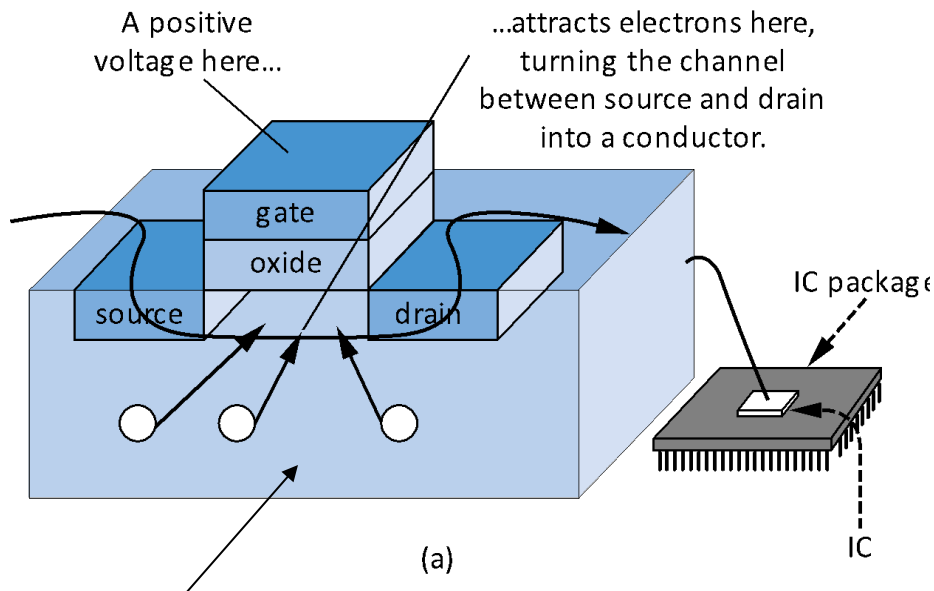


*An Intel Pentium processor IC
having millions of transistors*



The CMOS Transistor

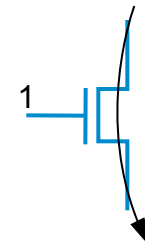
- CMOS transistor
 - Basic switch in modern ICs



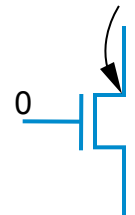
Silicon -- not quite a conductor or insulator:

Semiconductor

nMOS

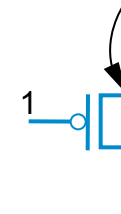
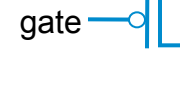


conducts

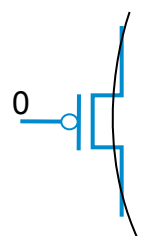


does not conduct

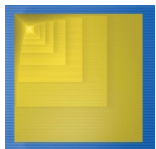
pMOS



does not conduct



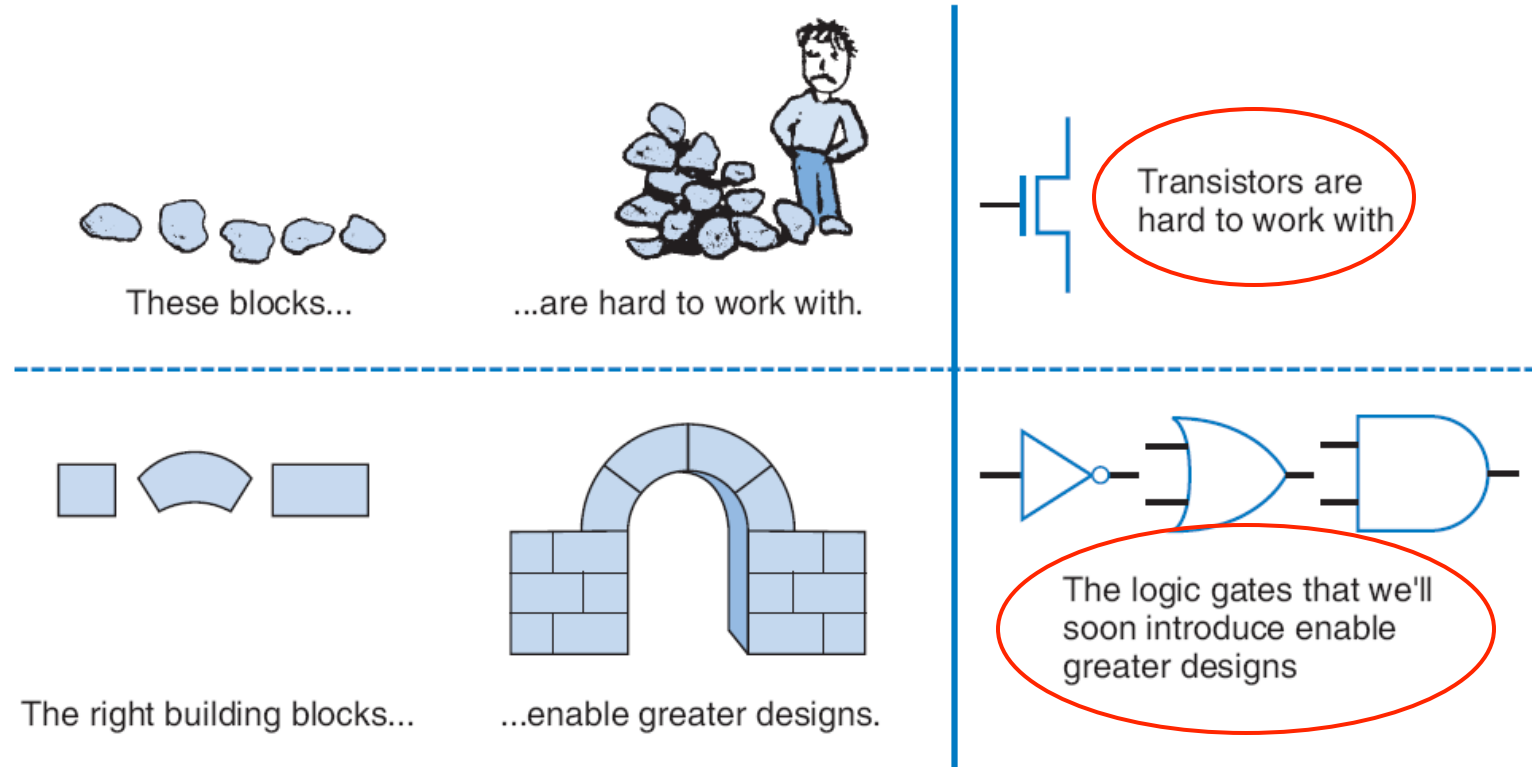
conducts



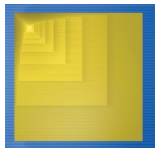
Boolean Logic Gates

Building Blocks for Digital Circuits

(Because Switches are Hard to Work With)



- “Logic gates” are better digital circuit building blocks than switches (transistors)
 - Why?...



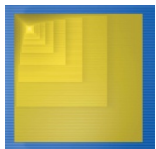
Boolean Algebra and its Relation to Digital Circuits

- To understand the benefits of “logic gates” vs. switches, we should first understand Boolean algebra
- “Traditional” algebra
 - Variable represent real numbers
 - Operators operate on variables, return real numbers
- **Boolean Algebra**
 - Variables represent 0 or 1 only
 - Operators return 0 or 1 only
 - Basic operators
 - AND: $a \text{ AND } b$ returns 1 only when both $a=1$ and $b=1$
 - OR: $a \text{ OR } b$ returns 1 if either (or both) $a=1$ or $b=1$
 - NOT: $\text{NOT } a$ returns the opposite of a (1 if $a=0$, 0 if $a=1$)

a	b	AND
0	0	0
0	1	0
1	0	0
1	1	1

a	b	OR
0	0	0
0	1	1
1	0	1
1	1	1

a	NOT
0	1
1	0



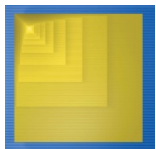
Boolean Algebra and its Relation to Digital Circuits

- Developed mid-1800's by George Boole to formalize human thought
 - Ex: “I'll go to lunch if Mary goes OR John goes, AND Sally does not go.”
 - Let F represent my going to lunch (1 means I go, 0 I don't go)
 - Likewise, m for Mary going, j for John, and s for Sally
 - Then **$F = (m \text{ OR } j) \text{ AND NOT}(s)$**
 - Nice features
 - Formally evaluate
 - $m=1, j=0, s=1 \rightarrow F = (1 \text{ OR } 0) \text{ AND NOT}(1) = 1 \text{ AND } 0 = \underline{0}$
 - Formally transform
 - $F = (m \text{ and NOT}(s)) \text{ OR } (j \text{ and NOT}(s))$
 - » Looks different, but same function
 - » We'll show transformation techniques soon

a	b	AND
0	0	0
0	1	0
1	0	0
1	1	1

a	b	OR
0	0	0
0	1	1
1	0	1
1	1	1

a	NOT
0	1
1	0



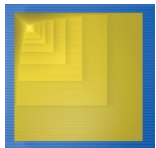
Evaluating Boolean Equations

- Evaluate the Boolean equation **$F = (a \text{ AND } b) \text{ OR } (c \text{ AND } d)$** for the given values of variables a, b, c, and d:
 - Q1: a=1, b=1, c=1, d=0.
 - Answer: $F = (1 \text{ AND } 1) \text{ OR } (1 \text{ AND } 0) = 1 \text{ OR } 0 = 1$.
 - Q2: a=0, b=1, c=0, d=1.
 - Answer: $F = (0 \text{ AND } 1) \text{ OR } (0 \text{ AND } 1) = 0 \text{ OR } 0 = 0$.
 - Q3: a=1, b=1, c=1, d=1.
 - Answer: $F = (1 \text{ AND } 1) \text{ OR } (1 \text{ AND } 1) = 1 \text{ OR } 1 = 1$.

a	b	AND
0	0	0
0	1	0
1	0	0
1	1	1

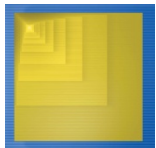
a	b	OR
0	0	0
0	1	1
1	0	1
1	1	1

a	NOT
0	1
1	0



Converting to Boolean Equations

- Convert the following English statements to a Boolean equation
 - Q1. *a is 1 and b is 1.*
 - Answer: $F = a \text{ AND } b$
 - Q2. *either of a or b is 1.*
 - Answer: $F = a \text{ OR } b$
 - Q3. *a is 0 and b is 0.*
 - Answer: $F = \text{NOT}(a) \text{ AND } \text{NOT}(b)$
 - Q4. *a is 1 and b is 0.*
 - Answer: $F = a \text{ AND } \text{NOT}(b)$



Converting to Boolean Equations

- Q1. A fire sprinkler system should spray water iff high heat is sensed and the system is set to enabled.
 - Answer: Let Boolean variable h represent “high heat is sensed,” e represent “enabled,” and F represent “spraying water.” Then an equation is: $F = h \text{ AND } e$.
- Q2. A car alarm should sound iff the alarm is enabled, and either the car is shaken or the door is opened.
 - Answer: Let a represent “alarm is enabled,” s represent “car is shaken,” d represent “door is opened,” and F represent “alarm sounds.” Then an equation is: $F = a \text{ AND } (s \text{ OR } d)$.
 - (a) Alternatively, assuming that our door sensor d represents “door is closed” instead of open (meaning $d=1$ when the door is closed, 0 when open), we obtain the following equation: $F = a \text{ AND } (s \text{ OR } \text{NOT}(d))$.

